LINUX PLUMBERS CONFERENCE (LPC)

Advancing PGO, AutoFDO, and Propeller in the Linux Kernel

Kernel Module Support • Profile Coverage & Layout Evaluation

PGO • Instrumentation

AutoFDO • Hardware PMU

Propeller • Code Layout

Rong Xu (xur@google.com) • Han Shen (shenhan@google.com) | Google

Agenda

PART 1

Kernel Module Support · Rong Xu

- Why modules matter, and the gaps in PGO, AutoFDO, and Propeller today
- One Kbuild interface for all three: <Mode>=gen and <Mode>=use
- AutoFDO and Propeller for .ko; PGO module support and the upstream path

PART 2

Profile Coverage & Layout Evaluation · Han Shen

- Why function-level coverage misleads once functions are split
- Line-level ground truth that holds up under inlining
- Cycles per Byte (CPB), a kernel case study, and the CI workflow

Part 1 brings the optimizations to modules → Part 2 measures how well they work

Motivation & Current Status

WHY KERNEL MODULES?

4–5%

of kernel CPU cycles on Google servers run in .ko modules, **even though most code is built-in**

~0.2% fleet-wide CPU savings opportunity

ANDROID (GKI)
Vendor kernel modules carry SoC and device drivers; many are performance-sensitive

Mode	Main Kernel (vmlinux)	Kernel Modules (.ko)
PGO	Out-of-tree (prior PR by Sami Tolvanen, rejected by Linus)	**BROKEN — No module profiles generated; built without PGO**
AutoFDO	Merged upstream in Linux 6.13	**UNSUPPORTED — Noted limitation in kernel documentation**
Propeller	Merged upstream in Linux 6.13	**UNSUPPORTED — No module profile conversion support**

Objective: Enable end-to-end profile generation, conversion, and Kbuild consumption for kernel modules across all three optimization modes.

Proposed Unified Kbuild Interface

Consistent build flags where <Mode> is PGO, AUTOFDO, or PROPELLER

1. Profile Generation Build

```
$ make LLVM=1 <Mode>=gen ...
```

- Passes profile generation flags to module builds
- Instruments modules (PGO) or enables required debug/section flags (AutoFDO/Propeller)
- Produces a separate profile file for each module

2. Profile Use Build

```
$ make LLVM=1 <Mode>=use \
    <Mode>_CLANG_PROFILE_DIR=<dir> ...
```

- PROFILE_DIR points to a flat directory containing generated profiles
- Kbuild automatically picks up matching module profile during each .ko compilation
- Unprofiled modules build normally

Why one profile per module, not one combined profile?

Each module (and vmlinux) has its own profile summary for hot/cold thresholds.

AutoFDO Today: The vmlinux Workflow

Upstream since Linux 6.13: profile a running kernel with perf, then rebuild with the profile

1. Build	2. Profile	3. Convert	4. Rebuild
Debug info for profiling; no runtime cost	Run a representative workload; perf samples taken branches	create_llvm_prof or llvm-profgen maps samples to source lines	Same config plus the profile; Clang optimizes vmlinux

```
# Intel LBR shown; AMD (BRS, LBRv2) and arm64 (SPE, ETM) use other events
$ scripts/config -e AUTOFDO_CLANG && make LLVM=1
$ perf record -e BR_INST_RETIRED.NEAR_TAKEN:k -a -N -b -c 500009 \
    -o <perf.data> -- <workload>
$ create_llvm_prof --binary=<vmlinux> --profile=<perf.data> \
    --format=extbinary --out=<vmlinux.afdo>
$ make LLVM=1 CLANG_AUTOFDO_PROFILE=<vmlinux.afdo>
```

Limitation: only vmlinux samples are converted, so modules get no profile data

AutoFDO: Module Profiles from One perf Run

All the work is in create_llvm_prof; no kernel changes are needed

1. Collect

- System-wide perf record (LBR) on the workload
- Stage the .ko files in one directory

2. Map Samples

- Attribute each sample to vmlinux or a module
- Find its function and source line in that binary

3. Emit Profiles

- vmlinux → <vmlinux.afdo>
- Modules with samples → <mod>.afdo in the same directory

```
# One command for vmlinux and all modules
$ create_llvm_prof --binary=<vmlinux> --profile=<perf.data> \
    --format=extbinary --out=<vmlinux.afdo> \
    --kernel_module_dir=<ko_dir>
```

AutoFDO for Modules: Two Complications

Mapping a sample to a source line needs the function's load address and its debug info

1. Replay the module loader's layout

- A .ko is ET_REL: its sections get addresses only when the module loader places them
- perf.data has one MMAP event per module, so create_llvm_prof replays the loader's layout (KernelModuleLayout)
- Especially with -ffunction-sections: every function is a separate section to place

2. Unstripped .ko files

- AutoFDO maps samples to source lines through debug info
- Stripped .ko files (e.g. INSTALL_MOD_STRIP=1) are of no use
- Keep the unstripped .ko files from the build tree

Either change would make the replay unnecessary:

- **perf: record each function's load address in perf.data (with -ffunction-sections)**
- **module.lds: group the function sections when linking the .ko**

Propeller: Same Approach as AutoFDO

The module changes mirror AutoFDO's, applied in Propeller's own tool

Same as AutoFDO

- Follows the same design: one perf run, per-module profiles
- Shares KernelModuleLayout (ET_REL section simulation) and MMAP sample matching

In Propeller's own tool

- Implemented in generate_propeller_profiles
- Emits per-module basic-block cluster and symbol order profiles into the target directory

```
# Consumed during the module build
$ make LLVM=1 PROPELLER=use PROPELLER_CLANG_PROFILE_DIR=<dir>
```

PGO: Per-Module Profiles in kernel/pgo

Implemented and working on the current kernel/pgo runtime: one profile per instrumented module

How It Works

• Discover each module's __llvm_prf_* sections: kallsyms at boot, module notifier on load/unload

• Per-module profraw and reset files in debugfs, plus reset_all and dump_all

• Modules share the vmlinux value-profiling (vnds) pool to limit memory

debugfs Layout

```
/sys/kernel/debug/pgo/
├── vmlinux/    profraw, reset
├── <module>/   profraw, reset
├── reset_all
└── dump_all
```

Limitation: the kernel carries its own copy of LLVM's profile format (~1,000 lines) and needs a patch for each format change; kernel/pgo stops at raw version 10, LLVM is at 11.

Share compiler-rt's Code with the Kernel ?

Builds on the module support: the kernel compiles compiler-rt's profile code, shipped with Clang

Reused from compiler-rt

Same code as user space, shipped with Clang

- Data structures (InstrProfData.inc)
- .profraw writer and reset
- Lock-free value-profiling hooks

LLVM cost: ~100–200 lines (est.)
Testing: existing compiler-rt tests
Precedent: Darwin kernel already uses the writer

Kernel After

Small and format-agnostic

- Section ranges and module registration
- Same debugfs files; a callback copies the bytes
- Builds the shared code with kernel flags
- No pgo_lock: NMI-safe value profiling

Kernel code (kernel/pgo/): ~1,000 → ~250 lines
Per format bump: ~50–60 → ~0 lines

Another option: binary correlation can simplify kernel PGO further, but value profiling support is unknown

Open Question for LPC: can the kernel build Clang-shipped compiler-rt code (Apache-2.0 license)?

Part 1: Summary & Discussion Topics

1. Kbuild UX	Feedback on unified Kbuild interface: <Mode>=gen, <Mode>=use, and <Mode>_CLANG_PROFILE_DIR=<dir>.
2. PGO Implementation	Module support API. Can the kernel compile compiler-rt's profile code shipped with Clang?
3. Module Layout	Should perf record each function's load address to avoid module layout simulation?

Up Next → Part 2: Profile Coverage & Layout Evaluation (Han Shen)

PART 2

Evaluating Profile Coverage & Code Layout in the Linux Kernel

Beyond Function-Level Coverage

Line-Level Ground Truth and Cycles per Byte (CPB)

Han Shen (shenhan@google.com) | Google

Motivation: Why We Need This Tool

A fast, deterministic evaluation of compiler optimizations and profile quality

1. Did the compiler optimize effectively

• Verify whether AutoFDO and Propeller code layout successfully placed hot execution paths into optimal memory sections (.text.hot).

2. Was the profile quality close to production

• Validate compiler profiles against real production perf.data

Core Value: A fast, deterministic feedback loop to validate compiler optimizations and profile quality before fleet rollout.

The Status Quo: Function-Level Coverage in the Kernel

The Traditional Profiling Analysis

• Run target kernel workload under perf (e.g. tcp_stream).

• Collect PMU samples, identify top functions taking 99%+ CPU cycles.

• Inspect compiled vmlinux symbol table to verify if function symbols reside in .text.hot.

The Function Recall Metric

Coverage = (Hot Functions in .text.hot) / (Total Hot Functions)

• **The Underlying Assumption:**
- Assumes whole functions are placed monolithically into sections.
- a higher score means better optimization or better profile quality

The Reality: Modern compilers and kernels do not treat functions as monolithic units anymore.

The Illusion of Function-Level Coverage

Massive Functions → Split Blocks

• **Functions group logic, not execution:**
  - Core routines (tcp_v4_rcv [~4 KB], tcp_rcv_established [~7 KB]) span thousands of bytes.

• **Sparse execution in production:**
  - Only 5%–10% of code paths execute on fast paths; 90%+ is cold error handling & cleanup.

• **Compiler response (Propeller/AutoFDO):**
  - Partitions functions at basic-block granularity:
    - Hot basic blocks -> **.text.hot**
    - Cold error paths -> **.text.split**

The Binary Evaluation Trap

• **The All-or-Nothing Fallacy:**
  - Function-level metrics assume functions are atomic units (100% covered or 0% covered).

• **The Binary Evaluation Trap:**
  - **Count as Covered?**
    Masks when hot blocks leak into .text.split, and overcounts cold code in .text.hot.
  - **Count as Missing?**
    Discards the 95%+ of hot execution successfully placed in .text.hot.

• **Result:** Function recall cannot measure basic-block layout.

Core Takeaway: Compilers optimize basic blocks, not functions. We must evaluate at instruction & line granularity.

Anatomy of a Split Line: Why One Source Line Exists in Two Sections

Compound Source Guard (core/skbuff.c:42): `if (skb && skb->len > 0)`

Compiled by LLVM + Basic-Block Splitting (Propeller) into separate fast and slow paths

Basic Block 1 & 2 (Fast Path)

DOMINANT EXECUTION PATH

```
BB1: test %rdi, %rdi; jz .Lsplit
BB2: cmpb $0, 0x8(%rdi); jle .Lsplit
```

Both checks pass during standard network packet processing

Placed in: `.text.hot (L1 Instruction Cache Resident)`

- Tightly packed with surrounding hot instructions for spatial locality

Basic Block 3 (Slow / Error Path)

COLD RECOVERY BRANCH

```
.Lsplit: callq kfree_skb
mov $0xffffffea, %eax; ret
```

Packet dropped or null pointer recovery branch

Evicted to: `.text.split (Far Memory Section)`

- Removed from hot cache lines to avoid polluting hardware L1i cache

DWARF .debug_line Mapping Resolution

```
PC range [0xffffffff812010..0xffffffff812028] in .text.hot   ──► maps to: core/skbuff.c:42
PC range [0xffffffff826040..0xffffffff826054] in .text.split ──► maps to: core/skbuff.c:42
```

KEY TAKEAWAY: The "Dual Presence" Phenomenon

- One source line, two physical sections: the hot path stays in .text.hot while cold error blocks are evicted to .text.split.
- Naive line matching breaks: checking line presence alone falsely flags split lines as unoptimized. We must evaluate instruction cycles.

Building an Instruction-Accurate Ground Truth

Horizontal Dataflow: Raw Hardware Samples → Smoothed CFG → DWARF Source Lines → 99.99% GT Cutoff

Input: PMU Profile

`perf.data`

- Hardware PMU cycles & branches
- tcp_stream workload

Sampling Gaps:
Interrupts every N events leave short fallthrough blocks unsampled.

SPARSE INTERRUPTS

Step 1: Propagate & Smooth

Basic Block Propagation

- Block freq = max(samples)
(eliminates block-length bias)

CFG Flow Smoothing

- Kirchhoff's flow conservation
Rescues 0-sample blocks
- All insts inherit smoothed weight

CONTINUOUS CFG FLOW

Step 2: Map to Source

DWARF Inline Stack

- Instruction PC Addr:
`0xffffffff812014`
- **Canonical 3-Tuple:**
`(Leaf, File, Line)`
- Unwinds inlined callers:
Distinguishes shared header inlines (spinlocks, atomics)
- Preserves instruction weight

CANONICAL 3-TUPLE

Step 3: Ground Truth

99.99% Cumulative Cutoff

- Aggregate weights per unique line

GT_lines (hot):

✓ ~9,200 Hot Lines
✓ 365,430 PMU Cycles

- Drops bottom 0.01% noise

GROUND TRUTH READY

KEY TAKEAWAY: Canonical Source-Line Representation

- Basic blocks propagate PMU samples; Kirchhoff CFG flow smoothing fixes sampling voids.
- DWARF unwinds inline stacks into Canonical 3-Tuples (Leaf, File, Line)—providing an unambiguous baseline for comparison.

Classifying Each Hot Source Line into Physical Text Sections

Input: 100.0% of Workload CPU Cycles (Ground Truth Hot Lines from perf.data)

1. Hot Core `.text.hot`	**Matched by 3-tuple (leaf, file, line) directly in .text.hot** • Captures Direct Caller Matches + Inlining Shifts in one clean step	**Primary Hot Core**
2. Pinned Text `.noinstr / .lock`	**Hardware-critical routines pinned by kernel linker script outside .text.hot** • native_sched_clock, _raw_spin_lock, entry/exit trampolines	**Architectural Text**

TOTAL OPTIMIZED TEXT (Hot Core + Pinned Text) = OVERWHELMING MAJORITY OF CYCLES

3. Split Text `.text.split`	**Cold basic blocks separated by Propeller (Dual Presence)** • Fast path executes in .text.hot; net split leakage to cold memory is negligible	**Negligible Leakage**
4. Unlikely (.text.unlikely)	Truly cold routines omitted from profile (e.g. error handling / cold setup)	**Truly Cold Paths**
5. Normal (.text)	General unpartitioned kernel routines (mutex_lock, mutex_unlock)	**Unpartitioned Text**

COMPLETE ATTRIBUTION: 100.0% of Workload Cycles Deterministically Classified (Zero Double-Counting)

The Bloat Fallacy: What Address Heatmaps Reveal

The Bloat Fallacy: What Address Heatmaps Reveal

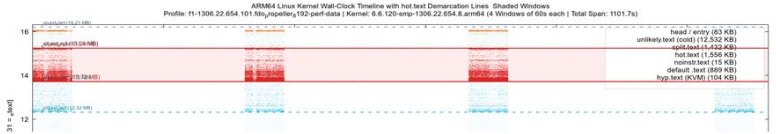
High cycle recall can mask severe spatial cache dilution; address heatmaps prompted our discovery of this flaw

Diluted Candidate Layout (Visible Gap in .text.hot)



- **Visible Horizontal Gap (Yellow Circle):** Cold space splits hot execution subbands
- Hot code fragmented across address space by cold blocks/padding -> L1i thrashing

Dense Baseline Layout (Compact & Continuous)



- **Solid, Unbroken Hot Band:** No apparent horizontal gap; continuous red core
- Basic blocks tightly packed into contiguous addresses, maximizing L1i spatial locality

The Gaming Problem & Math Trap of Precision

Dynamic Cycle Recall: `Recall = (Cycles in .text.hot) / (Total Cycles)`

- A compiler trivially gets 100% recall by dumping all kernel text into .text.hot!

Why Cycle Precision Cannot Catch Dead Code Bloat:

```
Precision = HotCycles / (HotCycles + ColdCycles)
```

- Cold code NEVER executes (Cold = 0) -> Precision = Hot / (Hot + 0) = 100% ALWAYS

- Cycle metrics only measure runtime execution; blind to dead code bloat.
- **Penalizing bloat requires measuring BYTES (physical cache footprint).**

The Masking Problem & Hardware Reality

The Deceptive Similarity on Paper:

Both configurations achieve virtually identical cycle recall on paper!

Finite Hardware L1i Cache Budget:

- Section bloat dilutes instruction cache lines with unexecuted instructions.
- Severe penalties: frequent L1i capacity evictions, iTLB thrashing, BTB conflicts.

Visual vs Automated Gating:

CI/CD cannot inspect heatmaps; we need a mathematical density metric!

THE SOLUTION: CYCLES PER BYTE (CPB)

Cold code costs physical memory space and cache lines, not CPU cycles. CPB divides hot cycles by section size in bytes, creating an ungameable density metric that automatically detects layout dilution in compiler CI/CD.

Cycles per Byte (CPB): Measuring .text.hot Execution Density

FORMAL DEFINITION: `CPB(.text.hot) = (Hot Cycles in .text.hot) / CodeSizeInBytes(.text.hot)`

Spatial Efficiency: How many CPU execution cycles reside in each physical byte of .text.hot in L1i cache?

High CPB: Densely Packed Instruction Cache Line

100% ACTIVE BYTES



Paying Maximum Cache Rent: Entire cache line contains active hot loop instructions. Maximizes L1i cache hit rate.

• Zero dead space • Optimal spatial locality • No cache line pollution

Diluted CPB: Cache Line Diluted by Cold Space

75% WASTED BYTES



Failing Cache Rent: Only 25% of the cache line is hot; 75% is unexecuted error handlers / padding / cold code.

• Footprint bloats section • Triggers frequent L1i cache line evictions & thrashing

RELATIVE CPB: `CPB(Candidate) / CPB(Baseline)`

> 1.0x: Denser Packing (PASS)

< 1.0x: Cache Dilution (BLOCK)

Real-World Evaluation: Kernel A (Baseline) vs. Kernel B (Candidate)

Workload: tcp_stream (9,268 hot source lines, 365,430 PMU cycles)

Section Metric	Kernel A (Baseline)	Kernel B (Candidate)	Change / Impact
Direct Caller Match (.text.hot)	68.11% cycles	70.92% cycles	+2.81% direct match
Total .text.hot Coverage	80.69% cycles	81.78% cycles	+1.09% cycle gain
Total Optimized Text	95.3% cycles	96.4% cycles	Includes .noinstr, .lock
Physical Size of .text.hot	584.8 KB	797.1 KB	+36.3% footprint expansion
Cycles per Byte (CPB)	**0.492 (1.00x)**	**0.366 (0.74x)**	**-26% spatial dilution**

- **Apparent Win (Cycle Coverage):** Kernel B captured +1.09% more hot cycles in .text.hot (81.78% vs 80.69%).
- **The Hidden Cost (Code Size):** Its .text.hot expanded by +36.3% (+212 KB of cache footprint).
- **The True Verdict (Density / CPB):** Execution density dropped by 26% (Relative CPB = 0.74x).
- **Actionable Outcome:** Warns compiler engineers that Kernel B's profile pulled in lukewarm code, providing the exact signal needed to tune Propeller's hotness threshold before rolling out to production.

Summary & Key Takeaways

Summary of Core Contributions

1. Beyond Function Granularity
- Basic-block splitting (AutoFDO / Propeller) requires instruction- and line-level evaluation.
- Resolves the binary evaluation trap and properly accounts for split hot vs. cold execution paths.

2. Deterministic 5-Category Cascading Attribution
- Partitions 100.0% of execution cycles into Hot, Pinned Text, Split, Unlikely, and Normal sections.
- Deterministically resolves deep compiler inlining shifts across different kernel binary builds.

3. Objective Function: Cycles per Byte (CPB)
- Pairing Dynamic Cycle Recall with Cycles per Byte (CPB) execution density prevents the Bloat Fallacy.
- Directly penalizes cache footprint expansion and preserves hardware L1i cache locality.

4. Fast & Static Developer Pre-Screen
- Pure in-memory analysis decodes ELF headers and DWARF quickly.
- Supplements dynamic benchmarking with an instantaneous score for kernel and APP developers.

Thank You! Questions & Discussion → Rong Xu & Han Shen